# 超算使用指南 (HPC Quick Start Guide)

## 目录

1. SSH 连接到超算
2. 基本 Linux 命令
3. 文件传输
4. 环境与模块管理
5. SLURM 作业调度系统
6. 提交批处理作业
7. 交互式节点 (salloc / srun)
8. GPU 作业
9. 作业监控与管理
10. 常用技巧

## 1. SSH 连接到超算

### 基本连接

```
ssh username@hpc.example.edu
```

### 使用特定端口

```
ssh -p 2222 username@hpc.example.edu
```

### 免密登录（SSH 密钥）

```
# 1. 在本地生成密钥对（如果还没有的话）
ssh-keygen -t ed25519 -C "your_email@example.com"

# 2. 将公钥复制到超算
ssh-copy-id username@hpc.example.edu

# 3. 之后直接连接即可，无需输入密码
ssh username@hpc.example.edu
```

### SSH 配置文件（推荐）

编辑本地 `~/.ssh/config`，之后只需 `ssh hpc` 即可连接：

```

```

```
Host hpc
    HostName hpc.example.edu
    User your_username
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    # 防止连接超时断开
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

**以下是我账号的配置，勿外传！！！**

```
Host CGE
  Hostname 202.114.0.141
  Port 21150
  User ud202180035
  ControlMaster auto
  ControlPath /tmp/ssh-%r@%h:%p
  ControlPersist 10m
  AddKeysToAgent yes
  IdentityFile ~/.ssh/CGE
  ServerAliveInterval 10
  ServerAliveCountMax 5
```

### 跳板机 / 代理连接

如果需要通过跳板机访问：

```
Host jump
    HostName jump.example.edu
    User your_username

Host hpc
    HostName hpc-internal.example.edu
    User your_username
    ProxyJump jump
```

---

## 2. 基本 Linux 命令

```
# 查看当前路径
pwd

# 查看磁盘配额使用情况（不同超算命令不同）
quota -s
df -h .
```

```
# 查看文件和目录
ls -lh          # 带详细信息和可读大小
ls -la          # 包括隐藏文件

# 目录操作
cd /path/to/dir
mkdir -p project/data    # 递归创建目录

# 文件操作
cp file1 file2           # 复制
mv file1 dir/            # 移动
rm file                  # 删除（谨慎使用）
rm -r directory/         # 递归删除目录

# 查看文件内容
cat file.txt             # 输出全部内容
head -n 20 file.txt      # 前 20 行
tail -n 20 file.txt      # 后 20 行
tail -f logfile.out      # 实时跟踪日志输出
less file.txt            # 分页查看（q 退出）

# 查找文件
find . -name "*.py"      # 按名称查找
find . -name "*.log" -mtime -1   # 最近 1 天修改的日志文件

# 文件内容搜索
grep "error" logfile.out          # 搜索关键词
grep -r "import torch" *.py      # 递归搜索 Python 文件

# 查看进程
top                      # 实时进程
htop                     # 更友好的进程查看器（如果可用）
ps aux | grep python     # 查找 Python 进程
```

---

## 3. 文件传输

### scp（小文件/少量文件）

```
# 上传文件到超算
scp local_file.py username@hpc.example.edu:/home/username/project/

# 上传整个目录
scp -r local_dir/ username@hpc.example.edu:/home/username/project/

# 从超算下载文件
scp username@hpc.example.edu:/home/username/results.tar.gz ./
```

### rsync（推荐，支持断点续传）

```
# 上传（同步本地到远程）
rsync -avz --progress local_dir/
username@hpc.example.edu:/home/username/project/

# 下载（同步远程到本地）
rsync -avz --progress username@hpc.example.edu:/home/username/results/
./results/

# 只同步更新的文件
rsync -avzu --progress local_dir/
username@hpc.example.edu:/home/username/project/
```

### 使用 SSH 配置简化

如果已配置 `~/.ssh/config`：

```
scp file.py hpc:/home/username/project/
rsync -avz data/ hpc:/home/username/project/data/
```

---

## 4. 环境与模块管理

### Module 系统

大多数超算使用 `module` 命令管理软件环境：

```
# 查看所有可用模块
module avail

# 搜索特定模块
module avail python
module avail cuda

# 加载模块
module load python/3.10
module load cuda/12.1
module load gcc/11.3

# 查看已加载模块
module list

# 卸载模块
module unload python/3.10

# 卸载所有已加载模块
module purge
```

```
# 查看模块信息
module show python/3.10
```

### Conda / Mamba 环境

```
# 加载 conda（视超算而定）
module load anaconda3
# 或
module load miniconda

# 创建新环境
conda create -n myenv python=3.10

# 激活环境
conda activate myenv

# 安装包
conda install numpy scipy matplotlib
pip install torch torchvision

# 从 yaml 文件创建环境（推荐，便于复现）
conda env create -f environment.yml

# 导出环境
conda env export > environment.yml

# 退出环境
conda deactivate
```

---

## 5. SLURM 作业调度系统

### 核心概念

| 概念 | 说明 |
|---|---|
| **Partition (分区)** | 不同类型的计算节点分组（如 `cpu`, `gpu`, `large`） |
| **Node (节点)** | 一台物理服务器 |
| **Task (任务)** | 一个进程 |
| **CPU** | 一个 CPU 核心 |
| **Job (作业)** | 用户提交的计算任务 |

### 查看集群信息

```
# 查看所有分区和节点状态
sinfo
```

```
# 更详细的节点信息
sinfo -N -l

# 查看特定分区
sinfo -p gpu

# 查看各分区的空闲资源
sinfo -o "%P %a %l %D %T %C"

# 查看节点详细配置
scontrol show node <nodename>

# 查看分区配置
scontrol show partition <partition_name>
```

---

## 6. 提交批处理作业

### 基本 SLURM 脚本模板

创建 `job.slurm` 文件：

```
#!/bin/bash
#SBATCH --job-name=my_job           # 作业名称
#SBATCH --partition=cpu             # 分区名称
#SBATCH --nodes=1                   # 节点数
#SBATCH --ntasks=1                  # 总任务数
#SBATCH --cpus-per-task=4           # 每个任务的 CPU 核数
#SBATCH --mem=16G                   # 内存
#SBATCH --time=02:00:00             # 最大运行时间 (HH:MM:SS)
#SBATCH --output=%x_%j.out         # 标准输出 (%x=作业名, %j=作业ID)
#SBATCH --error=%x_%j.err          # 标准错误输出

# ---- 环境设置 ----
module purge
module load python/3.10

conda activate myenv

# ---- 运行任务 ----
echo "Job started at $(date)"
echo "Running on node: $(hostname)"

python my_script.py

echo "Job finished at $(date)"
```

### 提交和管理作业

```
# 提交作业
sbatch job.slurm

# 提交并传递参数
sbatch --export=ALL,LR=0.001,EPOCHS=100 job.slurm

# 查看自己的作业队列
squeue -u $USER

# 查看作业详细信息
scontrol show job <job_id>

# 取消作业
scancel <job_id>

# 取消所有自己的作业
scancel -u $USER

# 查看作业历史记录
sacct -u $USER --starttime=2024-01-01 --
format=JobID,JobName,Partition,State,Elapsed,MaxRSS

# 查看作业效率报告
seff <job_id>
```

### Python 脚本作业示例

```
#!/bin/bash
#SBATCH --job-name=train_model
#SBATCH --partition=cpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=12:00:00
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
#SBATCH --mail-type=BEGIN,END,FAIL    # 邮件通知
#SBATCH --mail-user=you@email.com

# 确保日志目录存在
mkdir -p logs

module purge
module load python/3.10
conda activate myenv

echo "===== Job Info ====="
echo "Job ID: $SLURM_JOB_ID"
echo "Node: $SLURM_NODELIST"
```

```
echo "CPUs: $SLURM_CPUS_PER_TASK"
echo "Start: $(date)"
echo "===================="

python train.py \
    --epochs 100 \
    --batch-size 64 \
    --lr 0.001

echo "End: $(date)"
```

### 数组作业（Array Job）

同一个脚本跑多组参数：

```
#!/bin/bash
#SBATCH --job-name=sweep
#SBATCH --partition=cpu
#SBATCH --array=0-9                # 10 个子任务，ID 为 0-9
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH --output=logs/sweep_%A_%a.out   # %A=主作业ID, %a=数组索引

module purge
module load python/3.10
conda activate myenv

# 用数组索引选择不同的参数
LEARNING_RATES=(0.1 0.05 0.01 0.005 0.001 0.0005 0.0001 0.00005 0.00001
0.000005)
LR=${LEARNING_RATES[$SLURM_ARRAY_TASK_ID]}

echo "Array Task ID: $SLURM_ARRAY_TASK_ID, LR: $LR"

python train.py --lr $LR --output results/run_${SLURM_ARRAY_TASK_ID}
```

提交数组作业：

```
# 提交全部
sbatch array_job.slurm

# 只提交部分索引
sbatch --array=0-4 array_job.slurm

# 限制同时运行的子任务数（例如最多 3 个同时跑）
sbatch --array=0-9%3 array_job.slurm
```

# 7. 交互式节点 (salloc / srun)

> **重要**：不要在登录节点上直接跑计算任务！登录节点是共享的，应该只用于编辑文件、提交作业等轻量操作。

### salloc — 申请交互式资源

> **我们超算的分区名称**：CPU 分区为 `cpu-pgmf`，GPU 分区为 `gpu-pgmf`。

```
# 基本用法：申请 1 个节点，4 核，8G 内存，2 小时
salloc --nodes=1 --ntasks=1 --cpus-per-task=4 --mem=8G --time=02:00:00 --
partition=cpu-pgmf

# 申请 GPU 交互节点
salloc --nodes=1 --ntasks=1 --cpus-per-task=4 --mem=16G --time=01:00:00 --
partition=gpu-pgmf --gres=gpu:1

# 申请后会进入分配的节点，可以直接运行命令
# 退出交互节点
exit
```

`salloc` 成功后，你会得到一个 shell，可以在上面直接运行程序。

### srun — 直接运行交互式命令

```
# 直接在计算节点上运行一个交互式 shell
srun --nodes=1 --ntasks=1 --cpus-per-task=4 --mem=8G --time=02:00:00 --
partition=cpu-pgmf --pty bash

# 在计算节点上运行 Python
srun --ntasks=1 --cpus-per-task=4 --mem=8G --time=01:00:00 --
partition=cpu-pgmf --pty python my_script.py

# 在 GPU 节点上运行交互式 shell
srun --partition=gpu-pgmf --gres=gpu:1 --mem=16G --time=01:00:00 --pty
bash
```

### 交互式 Jupyter Notebook

```
# Step 1: 在超算上申请计算节点并启动 Jupyter
srun --partition=gpu-pgmf --gres=gpu:1 --mem=16G --cpus-per-task=4 --
time=04:00:00 --pty bash

# 进入计算节点后：
module load python/3.10
conda activate myenv
```

```
jupyter notebook --no-browser --port=8888 --ip=0.0.0.0

# Step 2: 在本地开一个 SSH 隧道
# 注意将 compute-node-name 替换为实际的计算节点名
ssh -N -L 8888:compute-node-name:8888 username@hpc.example.edu

# Step 3: 在本地浏览器打开
# http://localhost:8888/?token=<复制终端显示的 token>
```

---

## 8. GPU 作业

### 单 GPU 作业

```
#!/bin/bash
#SBATCH --job-name=gpu_train
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --gres=gpu:1                # 申请 1 块 GPU
#SBATCH --time=24:00:00
#SBATCH --output=logs/%x_%j.out

module purge
module load python/3.10
module load cuda/12.1

conda activate myenv

# 验证 GPU 可用
nvidia-smi
python -c "import torch; print(f'CUDA available:
{torch.cuda.is_available()}, Device: {torch.cuda.get_device_name(0)}')"

python train.py --device cuda
```

### 指定 GPU 型号

```
# 申请特定型号的 GPU（视超算配置）
#SBATCH --gres=gpu:a100:1           # 指定 A100
#SBATCH --gres=gpu:v100:2           # 指定 2 块 V100
```

### 多 GPU 作业（PyTorch DDP）

```
#!/bin/bash
#SBATCH --job-name=multi_gpu
#SBATCH --partition=gpu
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=4        # 每个节点 4 个任务（对应 4 个 GPU）
#SBATCH --cpus-per-task=8
#SBATCH --mem=128G
#SBATCH --gres=gpu:4               # 4 块 GPU
#SBATCH --time=48:00:00
#SBATCH --output=logs/%x_%j.out

module purge
module load python/3.10
module load cuda/12.1

conda activate myenv

# PyTorch DDP
export MASTER_ADDR=$(hostname)
export MASTER_PORT=29500

torchrun --nproc_per_node=4 train_ddp.py
```

多节点多 GPU 作业

```
#!/bin/bash
#SBATCH --job-name=multi_node_gpu
#SBATCH --partition=gpu
#SBATCH --nodes=2                  # 2 个节点
#SBATCH --ntasks-per-node=4        # 每个节点 4 个 GPU
#SBATCH --cpus-per-task=8
#SBATCH --mem=128G
#SBATCH --gres=gpu:4
#SBATCH --time=72:00:00
#SBATCH --output=logs/%x_%j.out

module purge
module load python/3.10
module load cuda/12.1

conda activate myenv

export MASTER_ADDR=$(scontrol show hostname $SLURM_NODELIST | head -n 1)
export MASTER_PORT=29500

srun torchrun \
    --nnodes=$SLURM_NNODES \
    --nproc_per_node=4 \
    --rdzv_id=$SLURM_JOB_ID \
    --rdzv_backend=c10d \
```

```
    --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \
    train_ddp.py
```

---

## 9. 作业监控与管理

### 实时监控

```
# 持续刷新自己的作业队列
watch -n 5 squeue -u $USER

# 查看作业输出（实时）
tail -f logs/my_job_12345.out

# SSH 到计算节点查看 GPU 状态（如果允许）
ssh compute-node-name nvidia-smi

# 持续监控 GPU
watch -n 2 nvidia-smi
```

### 作业状态码

| 状态 | 含义 |
|---|---|
| PD (PENDING) | 排队中 |
| R (RUNNING) | 运行中 |
| CG (COMPLETING) | 完成中 |
| CD (COMPLETED) | 正常完成 |
| F (FAILED) | 失败 |
| TO (TIMEOUT) | 超时 |
| CA (CANCELLED) | 被取消 |
| OOM (OUT_OF_MEMORY) | 内存不足 |

### 查看历史作业详情

```
# 查看过去一周的作业
sacct -u $USER --starttime=$(date -d '7 days ago' +%Y-%m-%d) \
    --
format=JobID,JobName,Partition,State,Elapsed,MaxRSS,MaxVMSize,NNodes,NCPUS

# 查看特定作业的资源使用情况
seff <job_id>
```

# 10. 常用技巧

## tmux / screen —— 保持会话

SSH 断开时任务不会丢失：

```
# 创建新的 tmux 会话
tmux new -s work

# 断开会话（不杀进程）
# 按 Ctrl+b, 然后按 d

# 重新连接
tmux attach -t work

# 列出所有会话
tmux ls
```

## 快速别名

在 `~/.bashrc` 中添加：

```
# SLURM 快捷命令
alias sq='squeue -u $USER'
alias si='sinfo -o "%P %a %l %D %T %C"'
alias sa='sacct -u $USER --
format=JobID,JobName%20,Partition,State,Elapsed,MaxRSS --starttime=$(date
-d "7 days ago" +%Y-%m-%d)'

# 交互式节点快捷命令
alias igpu='srun --partition=gpu-pgmf --gres=gpu:1 --mem=16G --cpus-per-
task=4 --time=02:00:00 --pty bash'
alias icpu='srun --partition=cpu-pgmf --mem=8G --cpus-per-task=4 --
time=02:00:00 --pty bash'
```

## 作业依赖

```
# 提交第一个作业
JOB1=$(sbatch --parsable preprocess.slurm)

# 第二个作业等第一个完成后再运行
sbatch --dependency=afterok:$JOB1 train.slurm

# 依赖类型:
# afterok:jobid    - 前一个作业成功后
# afternotok:jobid - 前一个作业失败后
```

```
# afterany:jobid   – 前一个作业结束后（不管成败）
# after:jobid      – 前一个作业开始后
```

### 常见问题排查

```
# 作业一直 PENDING? 查看原因
squeue -u $USER -o "%.18i %.9P %.8j %.8u %.2t %.10M %.6D %R"
# "R" 列显示原因:
#   (Priority)        – 排队等待优先级
#   (Resources)       – 等待资源释放
#   (QOSMaxJobsPerUserLimit) – 达到用户最大作业数

# 作业失败? 查看错误日志
cat logs/my_job_12345.err

# 内存不足 (OOM)? 增加 --mem 或优化代码
# 超时 (TIMEOUT)? 增加 --time
```

---

## 快速参考卡片

| 命令 | 用途 |
|---|---|
| `sbatch job.slurm` | 提交批处理作业 |
| `squeue -u $USER` | 查看自己的作业 |
| `scancel <job_id>` | 取消作业 |
| `sinfo` | 查看集群状态 |
| `salloc ...` | 申请交互式资源 |
| `srun --pty bash` | 直接获取交互式 shell |
| `sacct -u $USER` | 查看历史作业 |
| `seff <job_id>` | 查看作业效率 |
| `scontrol show job <id>` | 查看作业详细信息 |
| `module avail` | 查看可用软件模块 |
| `module load <name>` | 加载模块 |